Assignment 2: Test Automation Implementation

Deadline: Week 4

Objective

This assignment focuses on applying automated testing to the system and QA strategy you established in **Assignment 1**. You will:

- Implement automated tests for high-risk components.
- Introduce quality gates to ensure test reliability.
- Collect metrics and reports to feed directly into your research paper/project.

Learning Goals

By completing this assignment, you should be able to:

1. Select and implement automation tools suitable for your system type (web, mobile, API, desktop).
2. Integrate automated tests into a CI/CD pipeline for continuous verification.
3. Define **quality gates** (e.g., pass/fail thresholds, coverage requirements, error tolerance).
4. Collect and analyze metrics to measure test effectiveness and coverage.
5. Document automation strategy, CI/CD integration, and results for research purposes.

Connection to Assignment 1

- Builds on your **risk-based QA strategy** and testing environment from Assignment 1.
- Automates **high-risk modules/components** identified previously.
- Expands the **methodology section** of your research paper with automation details.
- Provides **initial experimental results** for use in Assignment 3.

Task / Assignment Component	By the end of this assignment, you will be able to… (Learning Outcome)	Bloom's Level	Where Demonstrated in Submission

Automated Test Implementation – Identify Test Scope	Identify high-risk modules and prioritize which functions require automation.	Understand / Analyze	Scope Table (Module/Feature, High-Risk Function, Test Priority, Notes)
Automated Test Implementation – Define Test Cases	Design detailed test cases with input, expected output, and scenario type for critical components.	Apply / Analyze	Test Cases Table (Test Case ID, Module/Feature, Input, Expected Result, Scenario Type, Notes)
Automated Test Implementation – Script Implementation	Implement maintainable, reusable automated scripts using the selected framework.	Apply / Create	Script Implementation Table (Script ID, Module/Feature, Framework, Location, Status, Comments)
Automated Test Implementation – Version Control Tracking	Track automation development progress and history via Git/GitLab commits.	Apply / Analyze	Version Control Table (Commit ID, Date, Module/Feature, Description, Author)
Automated Test Implementation – Evidence for Research Paper	Provide reproducible proof of automation results (screenshots, logs) for reporting metrics.	Apply / Evaluate	Evidence Table (Evidence ID, Module/Feature, Type, Description, File Location)
Quality Gate Definition & Integration – Define Pass/Fail Criteria	Set and justify thresholds for automation effectiveness, coverage, and reliability.	Evaluate / Apply	Quality Gate Table (Quality Gate ID, Metric, Threshold, Importance, Notes)

Quality Gate Definition & Integration – Integrate into CI/CD	Embed automated tests into CI/CD pipelines for continuous verification and alerting.	Apply / Create	CI/CD Pipeline Table & Screenshots/Diagrams (Steps, Tools, Trigger, Notes)
Quality Gate Definition & Integration – Alerting & Failure Handling	Define procedures for handling test failures, coverage issues, or pipeline errors.	Analyze / Evaluate	Alerting & Failure Handling Table (Scenario/Event, Alert Type, Recipient, Action, Notes)
Metrics Collection – Automation Coverage	Measure and report the proportion of high-risk modules that have automated tests.	Analyze / Evaluate	Coverage Table (Module/Feature, High-Risk Function, Automated?, Coverage %, Notes)
Metrics Collection – Execution Time Tracking (TTE)	Record and analyze the performance of automated tests, identifying bottlenecks.	Analyze / Evaluate	Execution Time Table (Module/Feature, Number of Test Cases, Execution Time, Total Time, Notes)
Metrics Collection – Defects vs Expected Risk	Compare detected defects with predicted risk levels to evaluate automation effectiveness.	Analyze / Evaluate	Defects Table (Module/Feature, High-Risk Level, Expected Defects, Defects Found, Pass/Fail, Notes)
Metrics Collection – Maintain Detailed Logs	Keep detailed logs of test runs, results, execution time, and defects for reproducibility.	Apply / Evaluate	Test Execution Log Table (Test Case ID, Module/Feature, Execution Date/Time, Result, Defects, Execution Time, Notes)

Metrics Collection – Metrics Reporting	Compile and visualize metrics in charts and tables to communicate test coverage and performance.	Analyze / Evaluate	Metrics Report Section (Bar/Line/Pie charts, tables, logs)
Documentation – Automation Approach & Tool Selection	Document rationale for automation tools and strategy, ensuring reproducibility.	Understand / Evaluate	QA Test Strategy Document (Automation Approach, Tool Selection, Scope, Reusability)
Documentation – Quality Gate Definitions	Report thresholds, observed results, and outcomes of quality gates.	Evaluate	QA Test Strategy Document (Quality Gate Tables & Results)
Documentation – CI/CD Integration Overview	Present pipeline steps, triggers, and automation integration clearly.	Apply / Create	QA Test Strategy Document (Pipeline Diagrams/Screenshots)
Documentation – Initial Results & Coverage Metrics	Present initial automation outcomes including coverage %, execution times, and defects found.	Analyze / Evaluate	QA Test Strategy Document (Tables, Charts, Evidence)
Documentation – Evidence for Reproducibility	Provide logs, screenshots, and code demonstrating rerun capability of automated tests.	Apply / Evaluate	QA Test Strategy Document / Evidence Folder (Screenshots, Logs, Code Snippets)

Suggested System-Specific Automation Tasks

System Type	Suggested Automation Focus	Recommended Tools/Frameworks	Detailed Guidance & Example Tasks
Web Application	- Login functionality- Form submissions- Workflow processes- API endpoints connected to the frontend	Selenium, Playwright	**Example Tasks:**• Automate login scenarios including valid/invalid credentials.• Automate form input validation, required fields, and error messages.• Automate multi-step workflows (e.g., checkout, user registration).• Automate API calls from the frontend and verify response codes and data.**Tips:**• Use page object models for maintainability.• Include negative testing to cover edge cases.• Parameterize tests to run with multiple data sets.
API / Microservices	- API endpoint validation- Error handling- Contract testing	Postman + Newman, REST Assured	**Example Tasks:**• Automate GET, POST, PUT, DELETE requests for critical endpoints.• Validate response codes, JSON schema, and payload correctness.• Automate authentication/authorization tests (e.g., JWT, OAuth).• Test error scenarios (invalid parameters, rate limits).**Tips:**• Include tests for edge cases and failure responses.• Maintain collections of test data for repeatable execution.• Integrate tests into CI/CD pipelines for continuous verification.

Desktop Application	- Core functions: file operations, UI interactions, data processing	Pywinauto, AutoIt	**Example Tasks:**• Automate file open/save dialogs, drag-and-drop actions.• Automate menu and toolbar interactions.• Validate output after core operations (calculations, data exports).• Include automation for batch operations or repetitive tasks.**Tips:**• Use object identifiers and coordinates carefully to avoid flaky tests.• Include tests for error dialogs and exception handling.• Parameterize inputs for repeated execution with different files or data.

General Tips for All System Types

1. Start with high-risk modules first — these provide the highest ROI for automation.
2. Maintain modular and reusable scripts — organize by features or components.
3. Document test steps and expected outcomes — useful for QA reports and research papers.
4. Include both positive and negative tests — verify that the system works as expected and fails gracefully.
5. Integrate with CI/CD pipelines — run tests automatically on each commit or release.

Assignment Tasks

1. Automated Test Implementation

Objective:
Implement automated tests for the **critical/high-risk modules** identified in Assignment 1. Ensure tests are **maintainable, repeatable, and well-documented**, with proper version control tracking.

Step 1: Identify Test Scope

List all high-risk modules and functions to be automated. Fill in the table below:

Module/Feature	High-Risk Function	Test Priority (High/Medium/Low)	Notes/Expected Outcome

Example: Login	User authentication	High	Must fail for invalid credentials

Step 2: Define Test Cases

Document test cases for each module/function. Include input, expected result, and scenario type.

Test Case ID	**Module/Feature**	**Description**	**Input Data**	**Expected Result**	**Scenario Type (Positive/Negative)**	**Notes**
TC01	Login	Valid login	username/password	Dashboard loads	Positive	N/A
TC02	Login	Invalid login	wrong password	Error message	Negative	Check UI display

Step 3: Track Script Implementation

Log details about each automated script written, including framework and status.

Script ID	**Module/Feature**	**Automation Framework**	**Script Name/Location**	**Status (Not Started/In Progress/Complete)**	**Comments**
S01	Login	Selenium	/tests/login_test.py	Complete	Includes positive & negative tests

Step 4: Version Control Tracking

Record commits and progress in Git/GitLab. This helps track development history.

Commit ID / Hash	**Date**	**Module/Feature**	**Description of Changes**	**Author**
abc123	29/03/2026	Login	Added positive login test case	Student Name
def456	30/03/2026	Login	Added negative login test case	Student Name

Step 5: Evidence for Research Paper

Track screenshots, logs, and notes for reproducibility and reporting metrics:

Evidence ID	**Module/Feature**	**Type (Screenshot/Log/Other)**	**Description**	**File Location/Link**
E01	Login	Screenshot	Dashboard after successful login	/evidence/login_success.png
E02	Login	Log	Failed login error log	/logs/login_fail.log

Deliverables

1. **Automated Test Scripts** – Scripts covering all high-risk modules, modular and well-commented.
2. **Version Control Repository Link** – Shows commits, progress, and organization of scripts.
3. **Filled Tables** – Scope, test cases, script implementation, version control, and evidence tables.

Best Practices

- Prioritize **critical modules first**, then expand to medium/low-risk modules.
- Ensure tests **run independently** and reset system state.
- Keep **detailed logs** for reproducibility and reporting metrics.
- Refactor scripts regularly for **clarity, maintainability, and efficiency**.

2. Quality Gate Definition & Integration

2. Quality Gate Definition & Integration

Objective:
Define and implement **quality gates** for your automated tests to ensure reliability, coverage, and performance. Integrate tests into a **CI/CD pipeline** with clear alerting and failure-handling procedures.

Step 1: Define Pass/Fail Criteria

Establish the thresholds and criteria that automated tests must meet to pass. Fill in the table below:

Quality Gate ID	Metric / Criterion	Threshold / Requirement	Importance (High/Medium/Low)	Notes
QG01	Code Coverage	≥ 80% of critical modules	High	Use coverage reports from framework
QG02	Critical Defects	0 critical defects allowed	High	Must block deployment if failed
QG03	Test Execution Time (TTE)	≤ 10 minutes per module	Medium	Adjust depending on module size
QG04	Regression Test Success	100% for critical workflows	High	Include end-to-end critical paths
QG05	Linting / Static Analysis	Zero major violations	Medium	Optional, adds code quality check

\

Step 2: Integrate Tests into CI/CD Pipeline

Automate execution of tests in your chosen CI/CD system:

Pipeline Step	Description	Tool / Framework	Trigger (On Commit / Scheduled / PR)	Notes

Step 1	Checkout code	GitHub Actions / Jenkins / GitLab CI	On commit to main branch	Ensure latest code is tested
Step 2	Install dependencies	npm / pip / gradle / Maven	Automatic	Use same environment as local tests
Step 3	Run automated tests	Selenium / Playwright / REST Assured / Appium	On commit / PR	Capture logs and results
Step 4	Generate test reports	HTML / XML / JSON reports	Automatic	Include coverage, execution time, failures
Step 5	Alerting / Notifications	Slack / Email / GitHub Status	On test failure	Include failure reason and link to logs

Step 3: Document Alerting & Failure Handling Procedures

Define how your pipeline reacts to failures and how the team should respond:

Scenario / Event	**Alert Type**	**Recipient / Channel**	**Action Required**	**Notes**
Critical test failure	Email + Slack	QA Lead, Dev Team	Investigate failure, rerun test, rollback if necessary	Include logs/screenshots
Coverage below threshold	Email	Dev Team	Update tests, fix gaps	Optional: block merge request
Test execution timeout	Pipeline log	DevOps	Optimize test scripts, rerun	Include time tracking
CI/CD pipeline error	Slack	DevOps	Check pipeline config, fix errors	N/A

Step 4: CI/CD Pipeline Documentation

Students should **capture screenshots or diagrams** of their pipeline including:

- Steps of the pipeline.
- Test execution status.
- Notifications or alerts setup.

Deliverables:

1. **Quality Gate Report** – Include definitions, thresholds, and results for each quality gate.
2. **CI/CD Pipeline Screenshots / Diagram** – Show pipeline integration of automated tests.
3. **Alerting & Failure Handling Documentation** – Explain how failures are detected, reported, and resolved.

Best Practices

- Automate **critical gates first**, then expand to medium/low-risk metrics.
- Ensure pipelines are **reproducible and version-controlled**.
- Include **logging and reporting** at each step to track failures, coverage, and execution time.
- Update quality gates and CI/CD configuration based on test results and system changes

3. Metrics Collection

Objective:
Collect and analyze metrics from your automated tests to evaluate **effectiveness, coverage, and performance**. These metrics will feed directly into your research paper/project.

Step 1: Track Automation Coverage

Measure the percentage of **high-risk modules/functions** that have automated test scripts.

Module/Feature	**High-Risk Function**	**Test Automated? (Yes/No)**	**Coverage %**	**Notes**
Login	User authentication	Yes	100%	Includes positive & negative cases
Checkout	Payment workflow	Yes	80%	Some edge cases not yet automated
Data Sync	Mobile sync module	No	0%	Planned for next iteration
API Endpoint A	GET request validation	Yes	100%	Includes error handling

Formula:
Automation Coverage (%) = (Number of automated high-risk functions / Total high-risk functions) × 100

Step 2: Track Execution Time (TTE)

Record how long it takes to execute your automated tests. Include both **total execution time** and **per-module time**.

Module/Feature	**Number of Test Cases**	**Execution Time per Test Case (sec)**	**Total Execution Time (sec)**	**Notes**
Login	5	12, 15, 10, 14, 11	62	Run on staging server
Checkout	8	20, 18, 22, 19, 21, 20, 23, 20	163	Includes API validations
API Endpoint A	4	8, 7, 9, 8	32	Run with Postman/Newman
Mobile Sync	6	25, 28, 30, 27, 26, 29	165	Emulated device tests

> **Tip:** Track execution time for both **CI/CD runs** and **local runs** to identify performance bottlenecks.

Step 3: Track Defects Found vs Expected Risk

Compare actual defects discovered during automation with the **expected risk levels** identified in Assignment 1.

Module/Feature	**High-Risk Level (High/Medium/Low)**	**Expected Defects**	**Defects Found**	**Pass/Fail**	**Notes**
Login	High	2	1	Pass	Detected incorrect error message
Checkout	High	3	2	Pass	Edge case for invalid card missing
Data Sync	Medium	1	0	Pass	No automation yet
API Endpoint A	High	2	2	Pass	All error scenarios detected

Tip: Use this table to discuss **automation effectiveness** in your research paper.

Step 4: Maintain Detailed Logs

For every test execution, keep a **log capturing**:

- Date/time of execution
- Test case ID / Script ID
- Result (Pass/Fail)
- Defects discovered
- Execution time per test
- Notes on failures or anomalies

Test Case ID	Module/Feature	Execution Date/Time	Result (Pass/Fail)	Defects Found	Execution Time (sec)	Notes
TC01	Login	29/03/2026 10:00	Pass	0	12	N/A
TC02	Login	29/03/2026 10:05	Fail	1	15	Invalid error message
TC03	Checkout	29/03/2026 10:15	Pass	0	20	N/A

Step 5: Metrics Reporting

- Compile **tables and charts** from Steps 1–4:
    - Bar chart: Automation coverage per module
    - Line chart: Test execution times per module
    - Table: Defects found vs expected risk

Deliverables:

1. **Metrics Report** – Include all tables, charts, and logs showing coverage, execution times, and defects.
2. **Evidence for Research Paper** – Screenshots of metrics dashboards or exported CSV/HTML reports from automation tools.

4. Documentation

Objective:
Update your **QA Test Strategy Document** to include all automation details, quality gates, CI/CD integration, and metrics. This will serve as a foundation for the **research paper's methodology and results sections**.

Step 1: Automation Approach & Tool Selection

Document your **automation strategy** and justify your choice of tools.

Section	Details to Fill In	Example / Prompt
Automation Approach	Describe your testing strategy (e.g., risk-based, regression-focused, end-to-end testing)	"Automate high-risk modules first using regression tests."
Tool Selection	List the tools/frameworks used and reason for selection	Selenium for web app due to browser compatibility; Postman + Newman for API tests due to easy integration in CI/CD.
Scope	Which modules/functions were automated	Login, Checkout, Data Sync, API Endpoint A
Reusability	How scripts are modular and maintainable	"Page Object Model used for web UI; reusable API helper functions."

Step 2: Quality Gate Definitions

Include the **pass/fail criteria** established for automated testing.

Quality Gate ID	Metric / Criterion	Threshold	Observed Results	Notes
QG01	Code Coverage	≥ 80%	85%	Passed, based on automated reports
QG02	Critical Defects	0	1 detected	Failure handled; rerun after fix
QG03	Test Execution Time	≤ 10 min/module	8 min	Optimized for faster execution
QG04	Regression Success	100%	100%	All critical workflows passed

---

Step 3: CI/CD Integration Overview

Document the CI/CD setup for automation.

Pipeline Step	Tool / Framework	Trigger	Description	Screenshot/Diagram
Step 1	GitHub Actions	On commit to main	Checkout latest code	[Insert Screenshot]
Step 2	Jenkins	Scheduled nightly	Run automated tests	[Insert Screenshot]
Step 3	Selenium / Playwright	On PR	Execute UI tests	[Insert Screenshot]
Step 4	Postman/Newman	On commit	Run API tests	[Insert Screenshot]
Step 5	Slack / Email	On failure	Alert team	[Insert Screenshot]

> **Tip:** Include workflow diagrams showing **pipeline steps**, **trigger points**, and **test execution flow**.

Step 4: Initial Results & Coverage Metrics

Include tables and charts summarizing **automation effectiveness**.

Module/Feature	Automated?	Coverage %	Execution Time (sec)	Defects Found	Pass/Fail
Login	Yes	100%	62	1	Pass
Checkout	Yes	80%	163	2	Pass
API Endpoint A	Yes	100%	32	2	Pass
Data Sync	No	0%	N/A	0	N/A

Visuals:

- Bar chart for automation coverage per module.
- Line chart for test execution times.
- Pie chart for defects vs expected risk.

Step 5: Evidence for Reproducibility

Include **logs, screenshots, and code snippets** demonstrating how the automation can be rerun.

Evidence ID	Module/Feature	Type (Screenshot / Log / Code)	Description	File Location / Link
E01	Login	Screenshot	Dashboard after successful login	/evidence/login_success.png
E02	Checkout	Log	Failed payment workflow log	/logs/checkout_fail.log
E03	API Endpoint A	Code snippet	Sample REST Assured test	/code/api_test.java

Deliverables

1. **Updated QA Test Strategy Document** – PDF or DOCX including:
    - Automation approach and tool rationale
    - Quality gate definitions and thresholds
    - CI/CD pipeline overview with screenshots and diagrams
    - Initial metrics from automation runs
2. **Reproducibility Evidence** – Logs, screenshots, and code snippets showing tests can be rerun.

Best Practices

- Keep **tables and charts updated** with each automation iteration.
- Ensure **screenshots and logs** clearly show test execution and results.
- Document **any failures and handling procedures** to support research methodology.
- Use this document as the **main reference for Chapter 3** of the research paper: "Automation Strategy and Implementation."

Connection to the Research Paper

Your assignment outputs will feed directly into your research paper/project:

- **Methods Section:** Document automation tools, approach, CI/CD integration, and quality gates.
- **Results Section:** Include baseline metrics from automated runs (coverage %, execution time, defects found).
- **Discussion Section:** Compare **expected risk vs actual defects**; analyze effectiveness of automation strategy.
- Include **screenshots, logs, and code snippets** as reproducibility evidence.

How This Builds Toward Final Project

- Provides **measurable results** for Assignment 3 (experimental analysis: performance and robustness).
- Supplies metrics and insights for Assignment 4 (research synthesis: trends, ROI, effectiveness).
- Demonstrates **practical value of automation** in QA research.

5. Deliverables Checklist

Objective:
Ensure all components of Assignment 2 are completed, documented, and ready for submission. This checklist also serves as a record of evidence for the research paper.

Deliverable	**Description**	**File/Location**	**Status (Not Started / In Progress / Complete)**	**Notes / Evidence**
Automated Test Scripts	Scripts for all high-risk modules/components, including positive and negative scenarios	/tests/ or GitHub repository link		Include naming conventions, comments, and modular structure
Updated QA Test Strategy Document	Document updated with automation approach, tool selection, quality gates, CI/CD overview, and initial results	PDF or DOCX		Include screenshots, workflow diagrams, and rationale for decisions
Quality Gate Report	Document containing defined pass/fail criteria, thresholds, and test results	PDF or DOCX		Include tables for code coverage, critical defects, execution time, and regression success

Metrics Report	Tables/charts showing automation coverage, execution times, defects found, and comparison to expected risk	PDF, Excel, or CSV		Include bar/line/pie charts where appropriate, with logs as evidence
CI/CD Pipeline Evidence	Screenshots or diagrams showing test integration, alerts, and pipeline execution	PNG, PDF, or embedded in Strategy Document		Include annotations or callouts for triggers, steps, and results

Notes for Students

- Use this checklist to **track progress for each deliverable** before the submission deadline.
- Include **links to GitHub/GitLab repositories** where possible to provide version-controlled evidence.
- Keep **all screenshots, logs, and diagrams organized** in a dedicated folder for easy reference in your research paper.
- Treat this checklist as part of your **research paper's reproducibility section**—demonstrating that your automation is documented, traceable, and verifiable.

AI Usage Policy

The use of Artificial Intelligence (AI) tools (e.g., ChatGPT, Copilot, or similar systems) is permitted in this assignment **only as a support tool**, not as a replacement for independent thinking and learning.

AI tools may be used as a **supporting learning assistant**, provided they enhance understanding rather than replace independent work. The following uses are permitted:

1. **Clarifying Concepts**
    - Using AI to better understand theoretical concepts such as:
        - Dynamic Connectivity,
        - Disjoint Sets,
        - Union–Find operations (makeSet, find, union),

        - Variants like Quick Find, Quick Union, Weighted Union, and Path Compression.
    - Asking AI for:
        - simplified explanations,
        - step-by-step breakdowns,
        - alternative ways of understanding difficult topics.
    - Students must **rephrase and reconstruct explanations in their own words** after understanding.
2. **Generating Ideas and Examples**
    - Using AI to:
        - explore possible real-world applications (e.g., network connectivity, social networks),
        - generate small illustrative examples to understand how algorithms behave,
        - get inspiration for diagrams or scenarios.
    - Any AI-generated idea must be:
        - **adapted, verified, and expanded** by the student,
        - integrated into a **personally developed explanation**.
3. **Improving Writing Quality**
    - Using AI tools to:
        - correct grammar and spelling,
        - improve clarity and coherence of explanations,
        - restructure sentences for better readability.
    - This includes editing:
        - definitions,
        - comparisons,
        - analytical explanations.
    - The **core content, reasoning, and structure must originate from the student**, not the AI.
4. **Debugging and Understanding Code**
    - Using AI to:
        - identify bugs in Union–Find implementations,
        - explain how specific parts of code work (e.g., path compression, union by rank),
        - suggest improvements or optimizations.
    - Students must:
        - understand every line of code submitted,
        - be able to **trace execution step-by-step**,
        - explain changes and justify why they improve performance.
5. **Checking Understanding (Self-Assessment)**
    - Using AI to:
        - quiz yourself on concepts,
        - test your explanations,

        - compare your reasoning with alternative approaches.
    - AI may be used as a **practice partner**, but not as the final answer generator

The following uses of AI tools are **strictly prohibited** and will be treated as violations of academic integrity:

1. **Submitting AI-Generated Content Without Understanding**
    - Providing answers, explanations, code, diagrams, or tables generated by AI without being able to clearly explain them.
    - Inability to justify:
        - how an algorithm works (e.g., Union–Find variants),
        - why a specific approach was chosen,
        - or how time complexity is derived.
    - If a student cannot defend their work during questioning, it will be assumed that the submission was not their own.
2. **Copy-Pasting AI Outputs**
    - Directly copying text, code, or structured answers (tables, definitions, comparisons) from AI tools with little or no modification.
    - Minor edits such as:
        - changing variable names,
        - reordering sentences,
        - or paraphrasing superficially

          do not count as original work.
    - Submissions must reflect **personal reasoning, structure, and explanation style**.
3. **Using AI to Complete the Entire Assignment**
    - Generating full solutions for all parts (A–E) using AI and submitting them as-is or with minimal edits.
    - Relying on AI to:
        - fill all tables,
        - generate all explanations,
        - solve all analytical tasks,
        - produce diagrams or pseudo-code without personal contribution.
    - This assignment is designed to assess **your analytical thinking and understanding**, which cannot be delegated to AI.
4. **Bypassing the Learning Process**
    - Using AI to skip essential steps such as:
        - analyzing differences between algorithms,
        - deriving time complexity,
        - constructing examples manually,

        - reasoning about optimizations (e.g., path compression, union by rank).
    - Over-reliance on AI that results in shallow or inconsistent understanding.
5. **Inconsistent or Non-Defensible Work**
    - Submissions that contain:
        - advanced explanations inconsistent with the student's demonstrated level,
        - incorrect or contradictory reasoning typical of unverified AI output,
        - code or logic the student cannot trace step-by-step.
    - During review or defense, students may be asked to:
        - explain intermediate steps,
        - modify their solution,
        - or solve a similar problem on the spot.

Consequences

- Suspected misuse of AI may result in:
    - additional oral questioning or defense,
    - partial or full loss of marks,
    - assignment resubmission,
    - or being reported under academic integrity policies (depending on severit

.

Student responsibility:

- You must be able to **explain and defend all parts of your submission** during evaluation.
- Any content generated with AI must be **critically reviewed, verified, and adapted**.
- If AI tools were used, you should **briefly acknowledge their use** (e.g., in a short note at the end of the assignment).

Example disclosure statement (optional):

> "AI tools were used to support understanding of Union–Find concepts and improve clarity of explanations. All final answers were reviewed and written by the student."

Rationale:

This policy ensures that students focus on **learning, problem-solving, and**

deep understanding, rather than passive completion of tasks. Proper use of AI enhances learning, while misuse undermines it.